

# AI Coding 實戰：Bun 創辦人用 AI 重寫 53 萬行程式

---

從技術可行性到 2041 年軟體工廠的演進



# 案例背景與核心事實

解析 Bun 創辦人 Jarred Sumner 的實戰經驗

# 實戰成果與技術亮點

- 1 使用 Claude Code 將 53 萬餘行 Zig 程式改寫為 Rust
- 2 僅用 11 天完成，傳統估計需 3 位資深工程師投入 1 年
- 3 最高同時啟動 64 支 Claude 平行協作，共計 6,502 次 Git 提交
- 4 導入 50 套 AI 工作流與對抗式審查機制

# 關鍵驗收數據

139萬次

結果檢查

透過 4,174 個測試檔與 6 萬+ 案例驗證

20%

執行檔瘦身

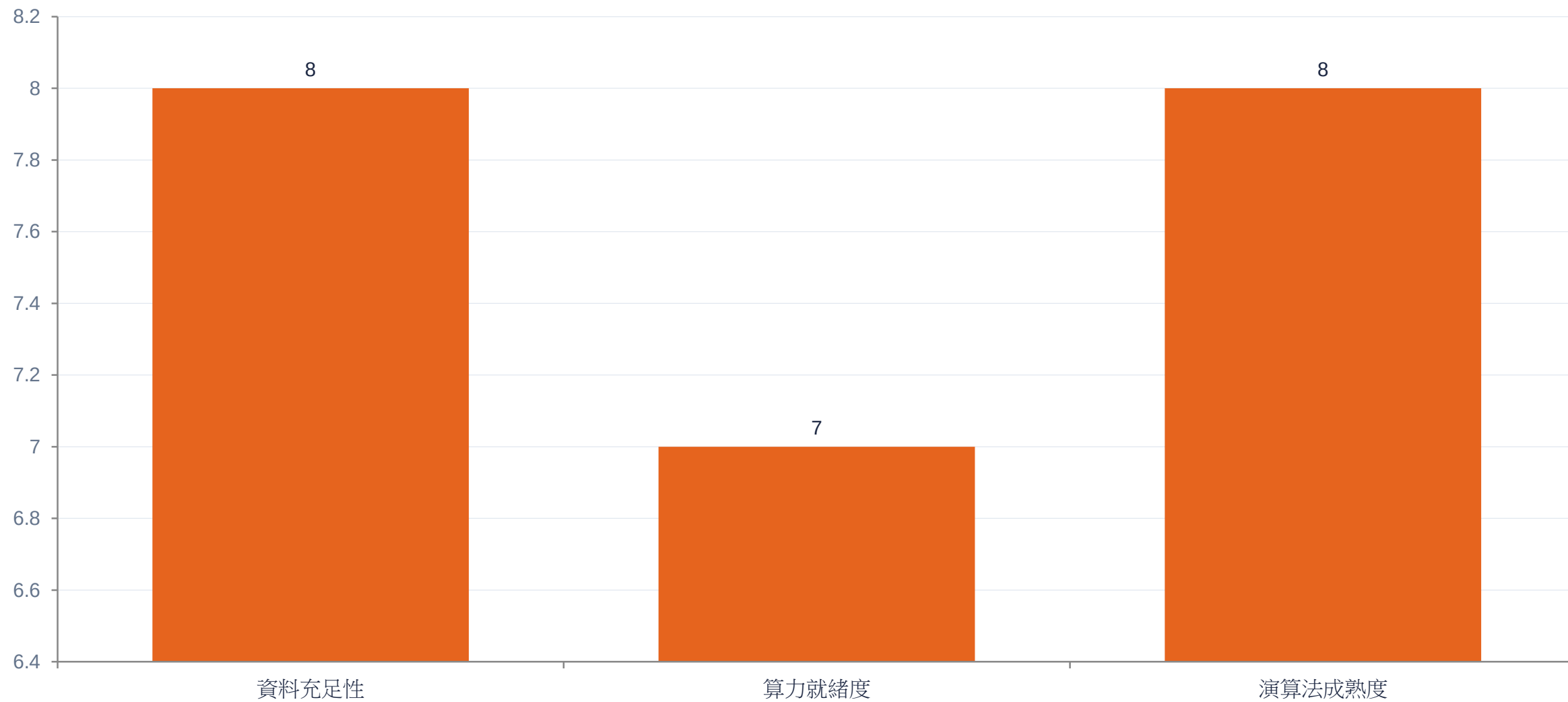
重寫後優化體積

16.5萬美元

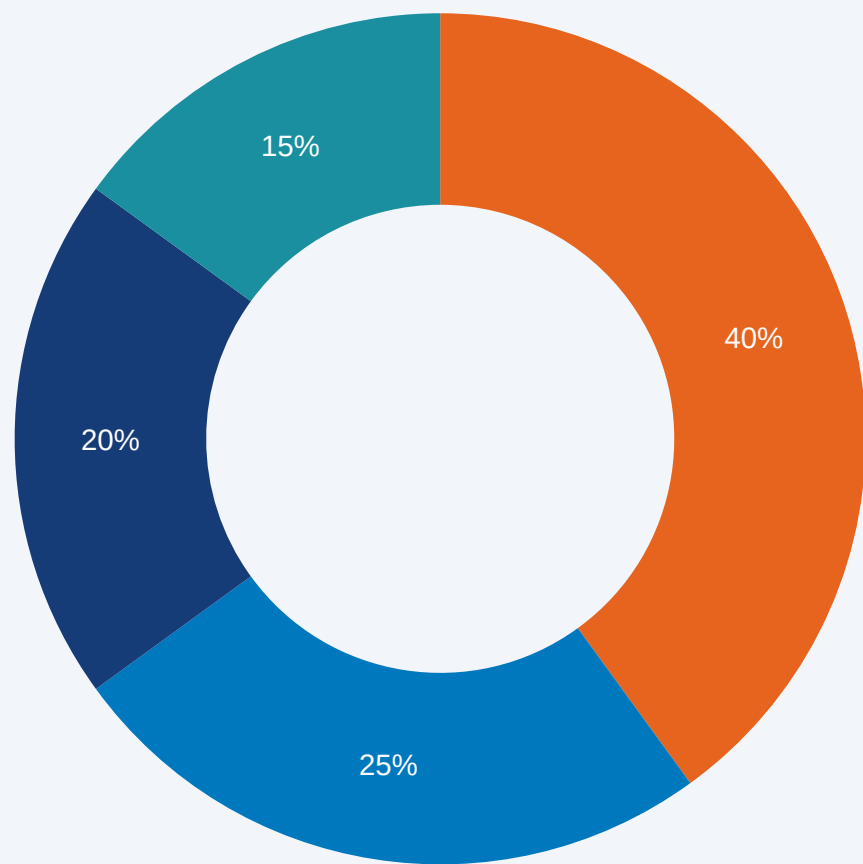
API 費用

本次大規模重寫的算力成本

# 技術可行性評分



# 最先受益行業分佈



■ 軟體開發工具 ■ 金融科技 ■ 電商/SaaS ■ 其他

## 關鍵洞察

軟體開發工具與金融科技核心系統是 AI Coding 最早落地的核心領域。

# 人類核心技能轉型



## 問題定義

定義轉換規則與規格



## Agent 编排

設計多 Agent 協作流程



## 風險裁決

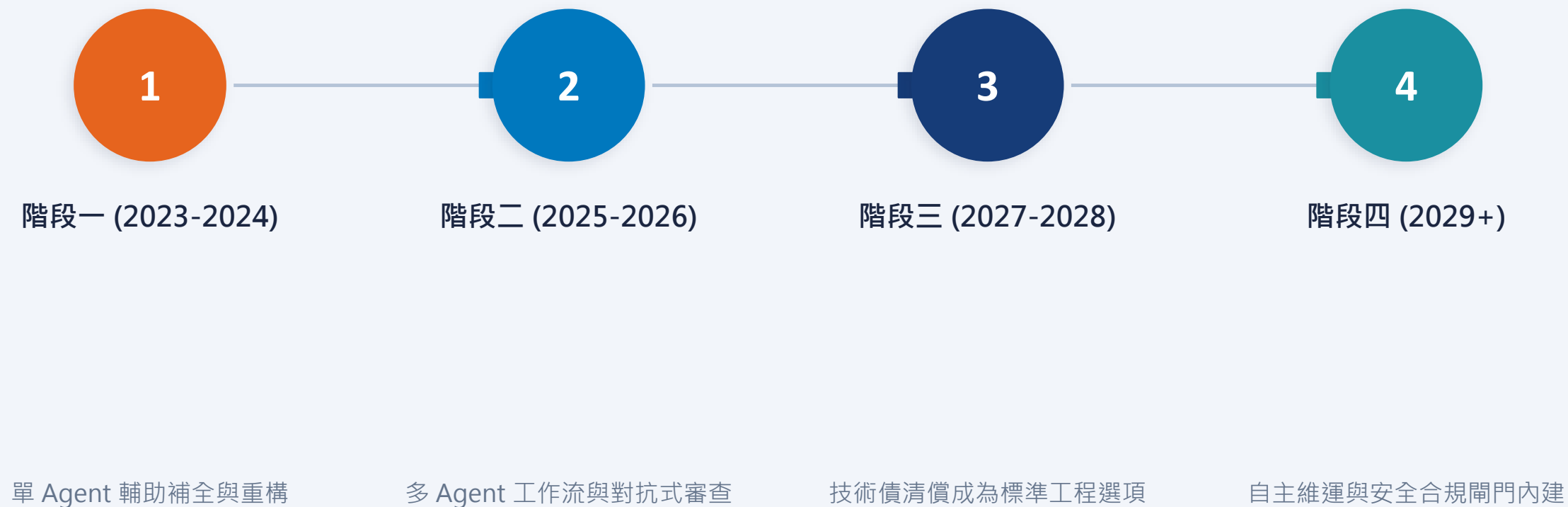
資安與 unsafe 審核



## 測試策略

設計自動化測試底座

# AI Coding 發展里程碑





# 人機協作分工模式

## AI 自動化 (75%)

- ◆ 程式碼改寫
- ◆ 編譯除錯
- ◆ 測試失敗迴圈
- ◆ 對抗式審查初篩

## 人類介入 (25%)

- ◆ 規則與品質閘門定義
- ◆ 可行性決策
- ◆ 最終品質把關
- ◆ 資安與 unsafe 審核

“  
AI Coding 降低大型重寫門檻，而非讓成本歸零。前提條件是成熟架構、大量自動測試、創辦人親自設計流程把關。”



# 總結與未來展望

2041 年的軟體工廠情景



# 核心結論與建議

- 1 軟體維運將轉向「多 Agent 工廠」產線，人類轉向架構與治理
- 2 必須將測試與對抗式審查作為硬閘門，避免過度信任 AI 產出
- 3 企業應優先投資於「問題定義」與「治理能力」，讓 AI 成為槓桿

# 謝謝聆聽

---

AI 深度研究系列